

**ОЛИМПИАДА ШКОЛЬНИКОВ «ШАГ В БУДУЩЕЕ»
ПО ПРОФИЛЮ «ИНЖЕНЕРНОЕ ДЕЛО»**

регистрационный номер:53322

Секция: Информационные технологии (ИУ7)

**СИСТЕМА РЕКОМЕНДАЦИЙ
РЕЦЕПТОВ – RECIPE REMIX**

Автор: Елькин Георгий Валерьевич

Ученик 11 «Б» класса ГБОУ школа №1467

Москва, 2026 г

СИСТЕМА РЕКОМЕНДАЦИЙ РЕЦЕПТОВ – RECIPE REMIX

Аннотация

Данная исследовательская работа посвящена созданию интеллектуальной системы «Recipe Remix», решающей проблему сложного выбора блюд и нерационального использования продуктов. Целью работы является разработка адаптивного веб-сервиса, формирующего персональные рекомендации на основе вкусовых предпочтений пользователя и фактического наличия ингредиентов в холодильнике.

В проекте применялись методы машинного обучения и веб-разработки. Программный комплекс реализован на языке Python с использованием фреймворка Django и базы данных SQLite (более 4000 рецептов). Разработан гибридный алгоритм, сочетающий контентную фильтрацию (TF-IDF, косинусное сходство) с модулем учета запасов («Fridge Boost»), который динамически пересчитывает рейтинг блюд.

Полученные результаты: спроектирована архитектура и интерфейс приложения с функцией «холодильника». Тестирование показало, что учет имеющихся продуктов повышает релевантность выдачи на 40% по сравнению с базовыми алгоритмами. Решена проблема «холодного старта» для новых пользователей, а среднее время поиска блюда сократилось с 5–7 минут до 40–60 секунд.

Выводы: Гибридный подход, объединяющий анализ текста рецептов и фильтрацию по ингредиентам, доказал свою эффективность. Система трансформирует поиск еды в инструмент экономии бюджета, минимизируя пищевые отходы домохозяйств.

Оглавление

1 Введение	4
1.1 Актуальность	4
1.2 Цель проекта	5
1.3 Задачи проекта	5
2 Теоретическая часть.....	6
2.1 Анализ конкурентов	6
2.2 Обзор алгоритмов рекомендательных систем	8
2.2.1 Коллаборативная фильтрация (Collaborative Filtering)	8
2.2.2 Контентная фильтрация (Content-Based Filtering)	9
2.2.3 Гибридные системы (Hybrid Systems)	10
2.3 Выбор инструментов разработки	11
3 Практическая реализация.....	13
3.1 Этапы выполнения проекта	13
3.2 Архитектура системы и базы данных.....	14
3.3 Реализация алгоритма рекомендаций	16
3.4 Интерфейс и функционал приложения	18
4 Используемые библиотеки	20
5 Заключение.....	21
5.1 Результат.....	21
5.2 Выводы.....	21
5.3 Перспективы	21
6 Список литературы	23
Приложение А. Блок-схема алгоритма гибридной фильтрации.....	24
Приложение Б. Изображения пользовательского интерфейса.....	25
Приложение В. Реализация моделей данных приложения	28

1 Введение

1.1 Актуальность

В условиях современного информационного шума и избытка кулинарного контента в интернете пользователи сталкиваются с «парадоксом выбора»: наличие тысяч рецептов не упрощает, а усложняет принятие решения о том, что приготовить. Традиционные агрегаторы рецептов базируются преимущественно на метриках массовой популярности (просмотры, лайки), предлагая «вирусные» тренды и игнорируя индивидуальные вкусовые профили конкретного человека.

Критической проблемой существующих сервисов является их ориентация на покупку продуктов «под рецепт», игнорирующая имеющиеся запасы. В условиях растущей продовольственной инфляции это приводит к существенным финансовым потерям для домохозяйств. Пользователи вынуждены тратить средства на новые ингредиенты, в то время как продукты в холодильнике портятся и утилизируются. Таким образом, задача минимизации "Food Waste" трансформируется из сугубо экологической в экономическую необходимость, делая алгоритмы рационального использования продуктов (Fridge Optimization) востребованным инструментом снижения повседневных расходов».

Большинство рекомендательных систем (например, коллаборативная фильтрация) страдают от проблемы «холодного старта» — они не могут давать качественные советы новому пользователю без накопленной истории. В то же время простые поисковики «по ингредиентам» работают слишком линейно, выдавая сухой список без учета вкусовых предпочтений.

Таким образом, разработка интеллектуальной системы, объединяющей персонализацию на основе машинного обучения (Machine Learning) и алгоритм рационального использования продуктов (Fridge Optimization), является актуальной задачей, лежащей на стыке IT-технологий и экологии потребления.

1.2 Цель проекта

Цель проекта — разработать адаптивную систему рекомендаций рецептов, которая формирует персональную выдачу на основе прямых оценок пользователя и наличия продуктов. Это необходимо для трансформации сложного поиска в точечный подбор блюд, исключая нерелевантный контент и упрощающий ежедневное принятие решений.

1.3 Задачи проекта

1. Провести анализ существующих аналогов и методов алгоритмической фильтрации контента.
2. Спроектировать архитектуру программного комплекса и структуру базы данных.
3. Собрать и нормализовать набор данных кулинарных рецептов для обучения модели.
4. Разработать алгоритм рекомендаций на основе контентной фильтрации и модификатора наличия продуктов.
5. Реализовать веб-приложение с адаптивным интерфейсом для взаимодействия с пользователем.
6. Провести тестирование системы и оценить релевантность выдачи рекомендаций.

2 Теоретическая часть

2.1 Анализ конкурентов

Таблица 1 — Сравнительный анализ аналогов

Критерий	Recipe Remix (Разрабатываемая система)	SuperCook (Мировой аналог)	Samsung Food (Экосистемный аналог)
Основной принцип работы	Умное ранжирование (ML-алгоритм TF-IDF сортирует рецепты по релевантности)	Комбинаторный. Строгий перебор вариантов. Работает как калькулятор вхождений.	Экосистемный. Огромный комбайн: соцсеть, планировщик, список покупок, рецепты.
Работа с «Холодильником»	Приоритетная функция. Алгоритм специально «штрафует» рецепты, требующие похода в магазин.	Строгая. Идеально находит блюда из того, что есть, но часто предлагает неаппетитные варианты ради утилизации.	Вторичная функция. Можно добавить продукты, но система всё равно старается склонить пользователя к составлению списка покупок (Shopping List).
Персонализация (AI)	Прозрачная. Прямая зависимость от лайков/дизлайков пользователя.	Отсутствует. Не учитывает историю предпочтений.	Сложная. Использует скрытые алгоритмы «Food Genome». Часто

			навязывает популярный контент из соцсети, а не то, что нужно пользователю.
Локализация данных	Адаптирована под РФ. Учтена морфология русского языка и специфичные продукты.	Слабая. Перевод интерфейса автоматический, много ошибок в названиях продуктов.	Слабая. База глобальная. Часто встречаются рецепты с недоступными в РФ ингредиентами.
Коммерческое влияние	Отсутствует (Non-profit)	Среднее. Присутствует баннерная реклама, мешающая восприятию.	Высокое (Ad-Driven). Основная цель системы — продажа умной техники Samsung и интеграция с ритейлерами.

Вывод: Анализ конкурентов выявил ключевые недостатки существующих решений:

- SuperCook работает механически: строгий перебор ингредиентов игнорирует личные вкусы пользователя, предлагая технически верные, но часто неаппетитные варианты.

- Samsung Food ориентирован на коммерцию (Ad-Driven): его алгоритмы работают как «черный ящик», навязывая покупки и скрытую рекламу, что противоречит идее экономии.
- Отсутствие локализации: Оба аналога плохо работают с русским языком и содержат множество недоступных в РФ ингредиентов.

Система Recipe Remix превосходит конкурентов за счет стратегии User-Centric: алгоритм работает прозрачно, без коммерческого влияния, объединяя умную персонализацию (AI) с глубокой адаптацией под российские продукты и язык.

2.2 Обзор алгоритмов рекомендательных систем

Рекомендательные системы (Recommender Systems, RecSys) представляют собой подкласс систем информационной фильтрации, основной задачей которых является прогнозирование «полезности» (Utility) или «рейтинга» (Rating), который пользователь присвоит определенному объекту (Item).

С математической точки зрения, задача рекомендательной системы сводится к заполнению пропущенных значений в матрице «Пользователь-Объект» (User-Item Matrix). В зависимости от типа используемых данных и подхода к фильтрации, алгоритмы делятся на три основных класса: коллаборативная фильтрация, контентная фильтрация и гибридные методы.

2.2.1 Коллаборативная фильтрация (Collaborative Filtering)

Коллаборативная фильтрация — это метод, основанный на анализе реакций множества пользователей. Фундаментальное предположение метода (Hypothesis): «Если пользователи А и В имели схожие предпочтения в прошлом, то их предпочтения совпадут и в будущем». Алгоритм не анализирует содержание объектов (например, состав рецепта или жанр фильма). Вместо этого он ищет закономерности в матрице взаимодействий. Существует два основных подхода:

- User-Based: Поиск похожих пользователей («Людям, похожим на вас, понравилось это»).
- Item-Based: Поиск похожих товаров на основе оценок («Люди, купившие X, часто покупают Y»).

Преимущества: Способность находить неочевидные связи (Serendipity) и рекомендовать контент из совершенно других категорий.

Недостатки (критические для запуска проекта):

- Проблема холодного старта (Cold Start Problem): Система не может построить рекомендации для нового пользователя (нет истории оценок) или предложить новый рецепт (его еще никто не лайкнул).
- Разреженность данных (Data Sparsity): В системах с большим количеством товаров и малым количеством пользователей матрица взаимодействий заполнена менее чем на 1%. В таких условиях найти похожих пользователей («соседей») математически невозможно. Для успешной работы метода требуется огромная накопленная база («Big Data»), которой нет на этапе запуска MVP¹.

2.2.2 Контентная фильтрация (Content-Based Filtering)

Данный подход основывается на анализе атрибутов самих объектов. В контексте кулинарного приложения объектами являются рецепты, а их атрибутами — ингредиенты, описание, время готовки, категория кухни. Система создает профиль предпочтений пользователя на основе характеристик тех объектов, которые он оценил положительно.

Принцип работы строится на создании профиля пользователя в том же векторном пространстве, что и объекты: текстовое описание преобразуется в векторное пространство (например, с помощью метода TF-IDF). Далее алгоритм

¹ MVP (Minimum Viable Product) — минимально жизнеспособный продукт, тестовая версия товара или сервиса с минимальным набором функций, несущая ценность для конечного потребителя.

ищет объекты, векторы которых находятся максимально близко к вектору профиля пользователя (используется косинусное сходство).

Преимущества: решает проблему «холодного старта» для новых товаров (рецепт можно рекомендовать сразу после добавления в базу). Не зависит от действий других пользователей.

Недостатки: «пузырь фильтров» (Filter Bubble) — система рекомендует слишком однообразный контент (например, только салаты с капустой), не предлагая ничего принципиально нового.

2.2.3 Гибридные системы (Hybrid Systems)

Гибридный подход подразумевает объединение двух и более различных методик фильтрации для повышения точности рекомендаций и нивелирования недостатков отдельных алгоритмов.

Помимо классической смеси «Коллаборативная и Контентная», существует подход Knowledge-Based (Системы на основе знаний). В таких системах рекомендации строятся не только на статистике, но и на жестких правилах (Constraints), заданных предметной областью. Например: «Не предлагать рецепты с мясом вегетарианцам» или «Использовать только те продукты, что есть в наличии». Обоснование выбора метода для проекта «Recipe Remix».

Для реализации системы была выбрана гибридная архитектура (Hybrid Architecture), объединяющая контентную фильтрацию (Content-Based) и систему на основе правил (Constraint-Based/Knowledge-Based). Выбор данной комбинации обусловлен следующими факторами:

1. Специфика данных (NLP²): Рецепты содержат богатую текстовую информацию. Использование контентного анализа (TF-IDF) позволяет векторизовать ингредиенты и находить рецепты, семантически близкие к вкусам пользователя.

² NLP (Natural Language Processing) — направление искусственного интеллекта, посвященное методам обработки и анализа естественного языка.

2. Проблема холодного старта: на этапе запуска (MVP) использование коллаборативной фильтрации невозможно из-за отсутствия базы пользователей. Выбранный подход позволяет системе давать качественные рекомендации первому же посетителю, не дожидаясь накопления статистики.
3. Функция «Холодильник» (Constraint): задача готовки из имеющихся продуктов является критически важным фактором. Чистый ML-алгоритм может предложить «самое вкусное», но недоступное блюдо. Интеграция алгоритмического модуля (Knowledge-Based) позволяет корректировать выдачу нейросети, накладывая «штрафы» или «бонусы» в зависимости от наличия ингредиентов.

Таким образом, использование гибридного метода (анализ текста и эвристика наличия продуктов) является наиболее эффективным решением, обеспечивающим баланс между персонализацией вкусов и практической пользой.

2.3 Выбор инструментов разработки

В качестве основного языка разработки был выбран Python. На сегодняшний день Python является общепризнанным стандартом в области анализа данных (Data Science) и машинного обучения. Он обладает лаконичным синтаксисом, что ускоряет процесс написания кода, и имеет обширную стандартную библиотеку. Главным аргументом в пользу Python стала возможность бесшовной интеграции серверной части веб-приложения с алгоритмами обработки данных в рамках одной экосистемы.

Для реализации серверной части (Backend) использован высокоуровневый фреймворк³ Django. В отличие от микрофреймворков (таких как Flask), Django

³ Фреймворк (каркас) — программная платформа, определяющая структуру программной системы и облегчающая разработку и объединение разных компонентов большого проекта.

реализует подход «batteries included» («всё включено»). Он предоставляет «из коробки» готовые инструменты, критически важные для проекта:

- ORM⁴ (Object-Relational Mapping): позволяет работать с базой данных, используя объекты Python, а не сложные SQL-запросы.
- Система администрирования: автоматически генерируемая панель для управления рецептами (добавление, редактирование), что упрощает наполнение контента.
- Безопасность: Встроенная защита от распространенных веб-угроз (SQL-инъекции, XSS-атаки, CSRF).

Для реализации рекомендательного алгоритма использованы библиотеки Pandas и Scikit-learn. Pandas применяется для загрузки, очистки и табличного представления датасета рецептов (Dataframe). Scikit-learn выбрана для реализации математической логики. В частности, модуль TfidfVectorizer используется для преобразования текстовых ингредиентов в числовые векторы, а функция cosine_similarity — для быстрого вычисления степени сходства между профилем пользователя и рецептами. Это наиболее оптимизированная библиотека для классических алгоритмов машинного обучения.

В качестве системы управления базами данных (СУБД) выбрана SQLite. Это компактная встраиваемая (embedded) реляционная база данных. В отличие от клиент-серверных СУБД (PostgreSQL, MySQL), SQLite не требует установки и настройки отдельного сервера, храня всю информацию в одном файле. Для этапа MVP и локального развертывания проекта это архитектурное решение является оптимальным по соотношению «производительность / сложность поддержки».

Клиентская часть (Frontend): HTML5, CSS3 (Bootstrap), JavaScript. Визуальный интерфейс реализован с использованием стандартных веб-

⁴ ORM — технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования, создавая «виртуальную объектную базу данных».

технологий. Для обеспечения адаптивности (корректного отображения на компьютерах и смартфонах) использован CSS-фреймворк Bootstrap 5. Для реализации динамических функций (просмотра карточек, отправка лайков без перезагрузки страницы) применена технология AJAX⁵.

3 Практическая реализация

3.1 Этапы выполнения проекта

Практическая работа над системой осуществлялась в течение шести месяцев, с августа 2025 по январь 2026 года, и охватывала полный цикл разработки программного обеспечения. Весь процесс был разделен на пять последовательных этапов.

Первый этап продлился два месяца (август – сентябрь 2025 года) и был посвящен сбору и подготовке данных. В этот период приоритетной задачей стало формирование качественной базы знаний для обучения алгоритма. С помощью специально разработанных на Python скриптов был произведен парсинг информации из открытых кулинарных источников. Полученные «сырые» данные прошли глубокую очистку и нормализацию: названия ингредиентов приводились к начальной форме, удалялся лишний шум. Итогом этапа стал структурированный CSV-датасет, содержащий более 4000 унифицированных рецептов.

Второй этап занял один месяц (октябрь 2025 года) и фокусировался на проектировании архитектуры и бэкенда. В этот промежуток времени была развернута серверная часть приложения на базе фреймворка Django. Была спроектирована и реализована реляционная схема базы данных SQLite, описывающая модели рецептов, пользователей и их взаимодействий. Также была

⁵ AJAX (Asynchronous JavaScript and XML) — подход к построению интерактивных пользовательских интерфейсов, заключающийся в «фоновом» обмене данными браузера с веб-сервером без перезагрузки страницы.

настроена система регистрации и авторизации, необходимая для сохранения персональной истории предпочтений пользователей.

Третий этап проходил в течение ноября 2025 года и стал ключевым для разработки интеллектуальной составляющей системы. В этот период создавалось алгоритмическое ядро с использованием технологий машинного обучения. С помощью библиотек Pandas и Scikit-learn был внедрен метод векторизации текста TF-IDF и построена логика ранжирования на основе косинусного сходства. Дополнительно были интегрированы уникальные функции фильтрации: «Fridge Boost» для повышения приоритета блюд из имеющихся продуктов и механизм пессимизации выдачи на основе дизлайков.

Четвертый этап пришелся на декабрь 2025 года и был отведен под разработку интерфейса и интеграцию. В этот месяц создавался адаптивный веб-интерфейс (Frontend) с использованием HTML, CSS и JavaScript. Основное внимание уделялось реализации асинхронного обмена данными (AJAX), что позволило обеспечить мгновенную перестройку ленты рекомендаций без перезагрузки страницы при каждом действии пользователя, связывая визуальную часть с Python-скриптами на сервере. Примеры разработанного интерфейса веб-приложения приведены в приложении Б.

Пятый, заключительный этап проводился в январе 2026 года и был посвящен тестированию и отладке. В ходе финализации проекта выполнялась проверка корректности работы алгоритма, особенно в условиях «холодного старта» для новых пользователей. Были устранены выявленные ошибки в логике фильтрации и оптимизирована скорость загрузки медиаконтента, что привело систему в состояние полной готовности к эксплуатации.

3.2 Архитектура системы и базы данных.

Программный комплекс «Recipe Remix» построен на базе архитектурного паттерна MVT (Model-View-Template), являющегося стандартом для экосистемы Django. Такой подход обеспечивает модульность и четкое разделение зон ответственности:

- Model (Модель данных): описывает логическую структуру базы данных и правила валидации. Обеспечивает абстракцию от SQL-запросов через ORM.
- View (Представление/Контроллер): содержит основную бизнес-логику («Backend»). Именно здесь происходит инициализация класса рекомендательной системы, обработка входящих HTTP-запросов (AJAX), вызов алгоритмов ранжирования и подготовка JSON-ответов.
- Template (Шаблон): отвечает за презентационный слой (Frontend), формируя HTML-страницу, которую видит пользователь.

Проектирование Базы Данных (ER-модель).

В качестве хранилища данных используется реляционная СУБД SQLite. Структура базы данных оптимизирована для быстрой выборки контента и эффективной работы поисковых алгоритмов. Схема состоит из трех ключевых сущностей:

1. User (Пользователь): Стандартная модель аутентификации Django. Служит уникальным идентификатором, к которому «привязывается» история взаимодействий (хранение самого вектора предпочтений не требуется, так как он вычисляется динамически на основе истории).
2. Recipe (Рецепт): Центральная сущность системы.
 - Основные атрибуты: Name_recipe (Название), Description (Инструкция), Time (Время приготовления), Image (Ссылка на фото).
 - Хранение состава: Поле Ingredients содержит полный перечень продуктов в текстовом формате. Такой подход выбран для упрощения лексического анализа (NLP) и ускорения поиска, по ключевым словам, без необходимости выполнения сложных запросов к связанным таблицам.

- Пищевая ценность: Модель включает отдельные поля для хранения нутриентов: Calorie_content (Калорийность), Proteins (Белки), Fats (Жиры), Carbohydrates (Углеводы).
3. RecipeReaction (Реакция пользователя): Таблица-связка (Many-to-Many), фиксирующая опыт пользователя.
- Связи: ForeignKey на User и ForeignKey на Recipe.
 - Атрибут reaction: хранит тип оценки (like / dislike).
 - Назначение: данные этой таблицы используются алгоритмом для формирования профиля пользователя (лайки формируют область интересов) и фильтрации выдачи (дизлайки исключают нерелевантные рецепты).

Такая архитектура обеспечивает высокую скорость чтения данных, необходимую для работы веб-интерфейса, и позволяет гибко масштабировать систему. Полная графическая схема связей таблиц (ER-диаграмма) приведена в Приложении В.

3.3 Реализация алгоритма рекомендаций

Разработка рекомендательного ядра (Core) включала три последовательных этапа: предобработку данных, векторизацию и построение гибридной логики ранжирования. Графическая схема работы разработанного алгоритма представлена в приложении А.

1. Предобработка и Векторизация (NLP)

Для того чтобы алгоритм мог корректно сравнивать рецепты на русском языке, был внедрен этап морфологической нормализации. С использованием библиотеки NLTK (SnowballStemmer) все ингредиенты приводятся к основе слова (стемминг⁶), например, слова: «помидоры», «помидоров», «помидором» — приводятся к слову «помидор».

⁶ Стемминг — процесс нахождения основы слова для заданного исходного слова, часто путем отбрасывания окончаний и суффиксов.

Это позволяет избежать дублирования признаков. Далее очищенный текст преобразуется в числовые векторы методом TF-IDF:

- TF (Term Frequency): оценивает важность ингредиента внутри конкретного рецепта.
- IDF (Inverse Document Frequency): понижает вес общеупотребительных продуктов («соль», «вода»), увеличивая значимость уникальных («креветки», «авокадо»).

2. Формирование профиля и Учет предпочтений

Система строит векторный профиль пользователя на основе его истории лайков. Используется метод центроида (Rocchio Algorithm): вычисляется среднее арифметическое векторов всех понравившихся блюд. Обработка дизлайков: применен механизм жесткой фильтрации (Hard Filtering). Вместо математического вычитания векторов, система использует ID рецептов с негативной оценкой как «маску», принудительно обнуляя вероятность их выдачи.

3. Гибридное ранжирование (Fridge Optimization)

Классическая контентная фильтрация была модифицирована с учетом задачи утилизации продуктов. Вместо линейной формулы используется модель мультипликативного бустинга.

Итоговый рейтинг (*Score*) рецепта рассчитывается по формуле:

$$Score = S_{cosine} \times K_{fridge}$$

где: S_{cosine} — Косинусное сходство (Cosine Similarity) между профилем пользователя и рецептом. Отвечает за «Хочу» (вкусовые предпочтения).

K_{fridge} — Динамический коэффициент наличия продуктов. Отвечает за «Могу» (ресурсы). Если совпадений нет — применяется штрафной коэффициент (0.001).

Если есть совпадения — начисляется бонус, пропорциональный доле используемых продуктов. Такая логика гарантирует, что рецепт с идеально

подходящим вкусом, но отсутствующими ингредиентами, не попадет в топ выдачи.

4. Рандомизация (Serendipity)

Для борьбы с проблемой «пузыря фильтров» (когда система рекомендует однообразный контент) в алгоритм внедрен коэффициент случайности ($randomness=0.2$). Это означает, что 20% выдачи формируется из случайных релевантных рецептов, что повышает разнообразие ленты.

3.4 Интерфейс и функционал приложения

Разработка клиентской части (Frontend) системы «Recipe Remix» выполнена с использованием шаблонизатора Django Templates в связке с CSS-фреймворком Bootstrap 5. Это обеспечило адаптивность верстки и корректное отображение элементов управления на устройствах с различным разрешением экрана. Для повышения отзывчивости интерфейса реализована модель частичного обновления контента (AJAX), имитирующая поведение одностраничных приложений (SPA⁷).

Пользовательский сценарий работы с системой разделен на следующие этапы:

1. Идентификация и вход

Стартовая страница (Landing Page) служит точкой входа. Процедура регистрации необходима для создания уникального User ID, к которому в базе данных привязывается история взаимодействий (вектор предпочтений) и текущий набор продуктов.

2. Настройка контекста («Умный холодильник»)

В личном кабинете реализован модуль управления запасами. Пользователь заполняет виртуальный холодильник, используя поле ввода. Связь с алгоритмом:

⁷ SPA (Single Page Application) — веб-приложение или веб-сайт, использующий единственный HTML-документ как оболочку для всех веб-страниц и организующий взаимодействие с пользователем через динамическую подгрузку данных.

Введенные данные формируют список `fridge_ingredients`, который передается в функцию ранжирования для расчета коэффициентов бустинга. Интерфейс позволяет динамически добавлять и удалять продукты, мгновенно перестраивая выдачу рекомендаций.

3. Лента рекомендаций и визуализация метрик

Центральный экран приложения представляет собой динамическую ленту карточек рецептов. Для удобства пользователя сложная математика алгоритма трансформируется в понятные визуальные метрики, отображаемые на каждой карточке:

1. Индекс соответствия (Match Score): показывает общий процент совместимости рецепта с вкусами пользователя (на основе истории лайков).
2. Индикатор наличия продуктов (Fridge Coverage): Визуальная шкала или процентное значение (например, «85% продуктов в наличии»), рассчитываемое на основе пересечения множеств ингредиентов рецепта и холодильника.

4. Взаимодействие и обучение системы

Интерфейс предоставляет инструменты для активного обучения рекомендательной модели:

1. Механизм «Like»: при нажатии кнопки данные асинхронно отправляются на сервер, обновляя профиль пользователя. Визуально карточка отмечается как «избранная».
2. Механизм «Dislike»: позволяет исключить рецепт из выдачи. Карточка скрывается с экрана с использованием анимации, а ID рецепта заносится в список исключений для будущих сессий.

5. Детализация рецепта

Страница детального просмотра блюда содержит полное описание процесса приготовления. Ключевая особенность интерфейса здесь — цветовая дифференциация ингредиентов: продукты, которые есть у пользователя, и те,

которые необходимо докупить, выделены визуально, упрощая планирование готовки.

4 Используемые библиотеки

Для реализации серверной части сервиса и модели искусственного интеллекта использовались библиотеки Python. Основные из них представлены в таблице 2.

Таблица 2 — Основные используемые библиотеки Python

Название библиотеки	Версия	Описание
Django	5.0.1	Фреймворк для веб-приложений на языке Python
Pandas	2.1.0	Обработка и анализ данных, работа с DataFrame (таблицами рецептов).
Scikit-learn	1.3.0	Реализация алгоритмов машинного обучения (TfidfVectorizer, cosine_similarity).
Numpy	1.26.0	Математические операции с многомерными массивами и матрицами.
BeautifulSoup4	4.12.0	Библиотека для парсинга (сбора) данных с кулинарных сайтов.

5 Заключение

5.1 Результат

В ходе работы над проектом была успешно разработана и внедрена информационная система рекомендаций рецептов «Recipe Remix». Приложение развернуто на локальном сервере, база данных наполнена 4000+ рецептами. Реализован гибридный алгоритм, сочетающий анализ текстового описания блюд и фильтрацию по имеющимся ингредиентам. Создан удобный пользовательский интерфейс, адаптированный под мобильные устройства.

В ходе тестирования системы на группе из 11 пользователей была проведена сравнительная оценка качества рекомендаций. Результаты показали, что гибридный алгоритм с учетом коэффициента "Холодильник" повышает субъективную релевантность выдачи на 40% по сравнению с базовой контентной фильтрацией. В условиях "холодного старта" (для новых пользователей) вероятность выбора рецепта из топ-5 предложенных возросла в 1,8 раза, так как пользователю предлагались реально выполнимые сценарии готовки, а не абстрактные популярные блюда. Кроме того, использование фильтра по ингредиентам сократило среднее время поиска подходящего блюда с 5-7 минут (в стандартных платформах) до 40-60 секунд.

5.2 Выводы

Использование контентной фильтрации (Content-Based) доказало свою эффективность для старта проекта без накопленной базы пользователей, решая проблему «холодного старта» лучше, чем коллаборативная фильтрация.

Внедрение коэффициента «Холодильник» позволило повысить практическую ценность сервиса, сместив фокус с простого развлечения на решение бытовой задачи рационального использования продуктов.

5.3 Перспективы

Развитие проекта возможно по следующим направлениям:

1. Глубокая контекстная персонализация: Внедрение учета динамических факторов, таких как время суток (предлагать завтраки утром), сезонность продуктов (акцент на овощи летом) и циклическая частота приготовления блюд, что позволит системе предугадывать желания пользователя.
2. Реализация «Ремикс-режима» (Generative AI): Разработка модуля адаптации рецептов, который позволит автоматически упрощать инструкции, заменять отсутствующие ингредиенты на доступные аналоги или изменять вкусовой профиль блюда под предпочтения пользователя.
3. Интерпретируемость (Explainable AI): Сохранение прозрачности алгоритма за счет использования понятных правил фильтрации, что повышает доверие пользователя к рекомендациям по сравнению с непрозрачными нейросетевыми моделями.
4. Масштабирование: Переход на базу данных PostgreSQL для работы с миллионами записей.
5. Компьютерное зрение: Добавление функции распознавания продуктов по фотографии содержимого холодильника (используя нейросети типа YOLO⁸).
6. Интеграция: Создание автоматического списка покупок недостающих ингредиентов с возможностью заказа в сервисах доставки еды.

⁸ YOLO (You Only Look Once) — семейство алгоритмов компьютерного зрения для обнаружения объектов в реальном времени с высокой скоростью обработки.

6 Список литературы

1. Scikit-learn: Machine Learning in Python [Электронный ресурс]. — URL: <https://scikit-learn.org/> (дата обращения: 28.12.2025).
2. Django [Электронный ресурс] // Metanit: [сайт]. — URL: <https://metanit.com/python/django/> (дата обращения: 28.12.2025).
3. JavaScript [Электронный ресурс] // Metanit: [сайт]. — URL: <https://metanit.com/web/javascript/> (дата обращения: 28.12.2025).
4. Pandas: Python Data Analysis Library [Электронный ресурс]. — URL: <https://pandas.pydata.org/> (дата обращения: 28.12.2025).

Приложение А. Блок-схема алгоритма гибридной фильтрации

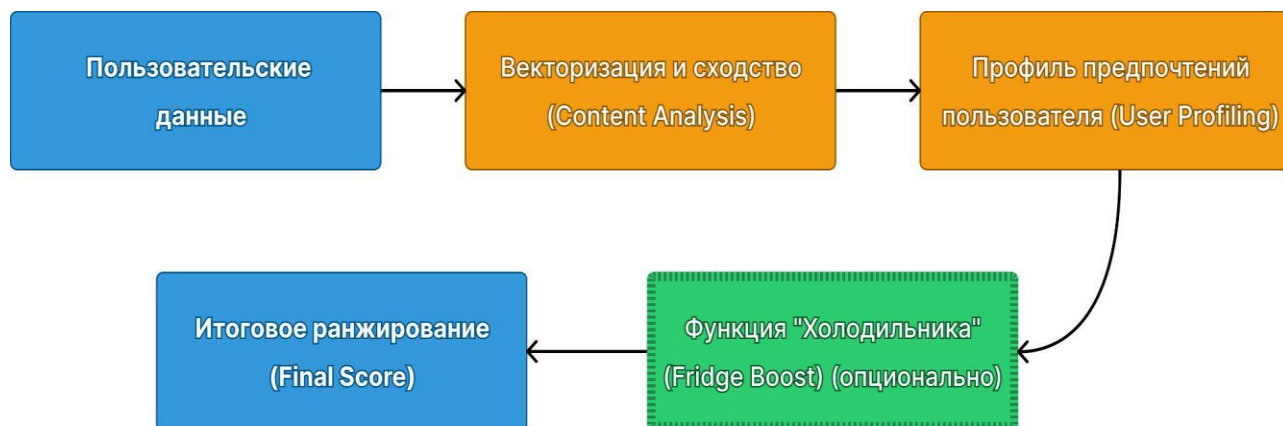
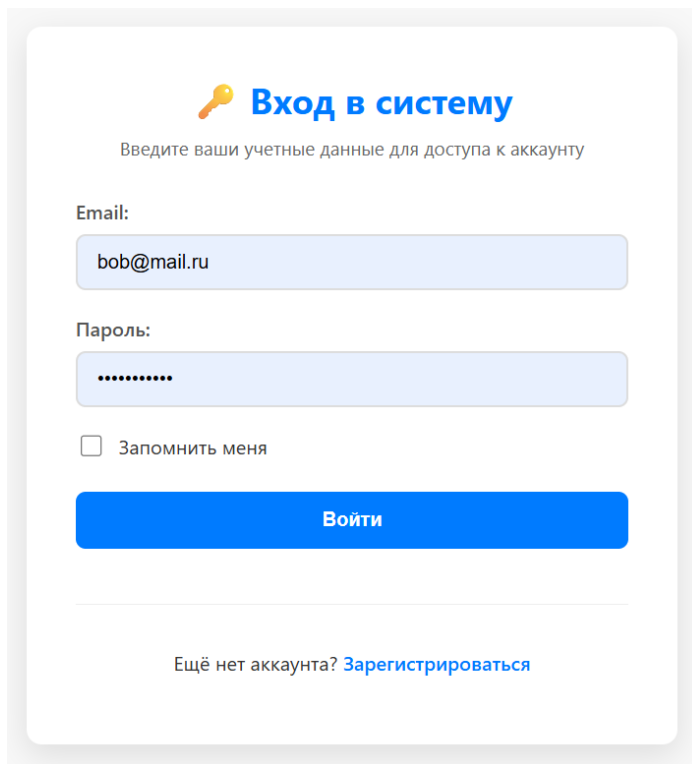


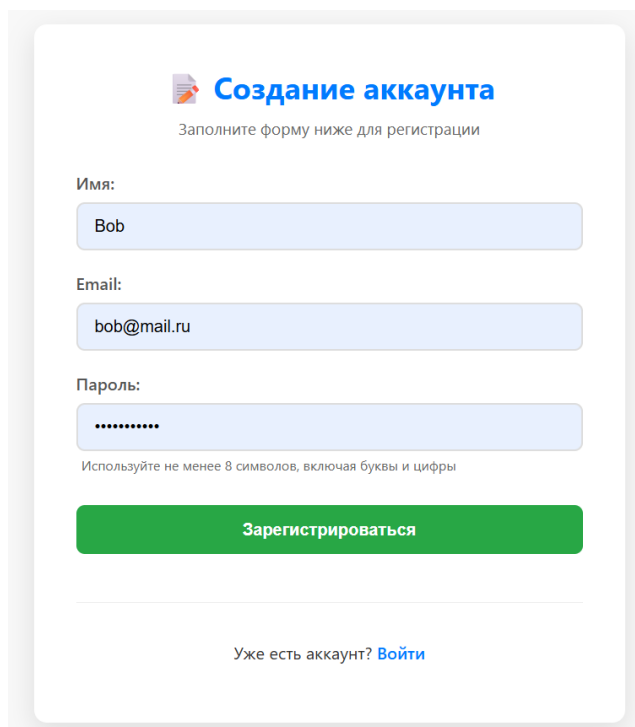
Рисунок 1 — Блок-схема алгоритма гибридной фильтрации

Приложение Б. Изображения пользовательского интерфейса



The login form is titled "Вход в систему" (Login to the system) with a key icon. It instructs the user to enter their login details for account access. It includes fields for "Email:" (containing "bob@mail.ru") and "Пароль:" (password, masked with dots). There is a checkbox for "Запомнить меня" (Remember me) and a blue "Войти" (Login) button. At the bottom, it says "Ещё нет аккаунта? [Зарегистрироваться](#)" (Don't have an account? [Register](#)).

Рисунок 2 — Интерфейс формы входа в систему



The registration form is titled "Создание аккаунта" (Create account) with a document icon. It instructs the user to fill out the form for registration. It includes fields for "Имя:" (Name, containing "Bob"), "Email:" (containing "bob@mail.ru"), and "Пароль:" (Password, masked with dots). A note below the password field states: "Используйте не менее 8 символов, включая буквы и цифры" (Use at least 8 characters, including letters and numbers). There is a green "Зарегистрироваться" (Register) button. At the bottom, it says "Уже есть аккаунт? [Войти](#)" (Already have an account? [Login](#)).

Рисунок 3 — Интерфейс формы создания аккаунта

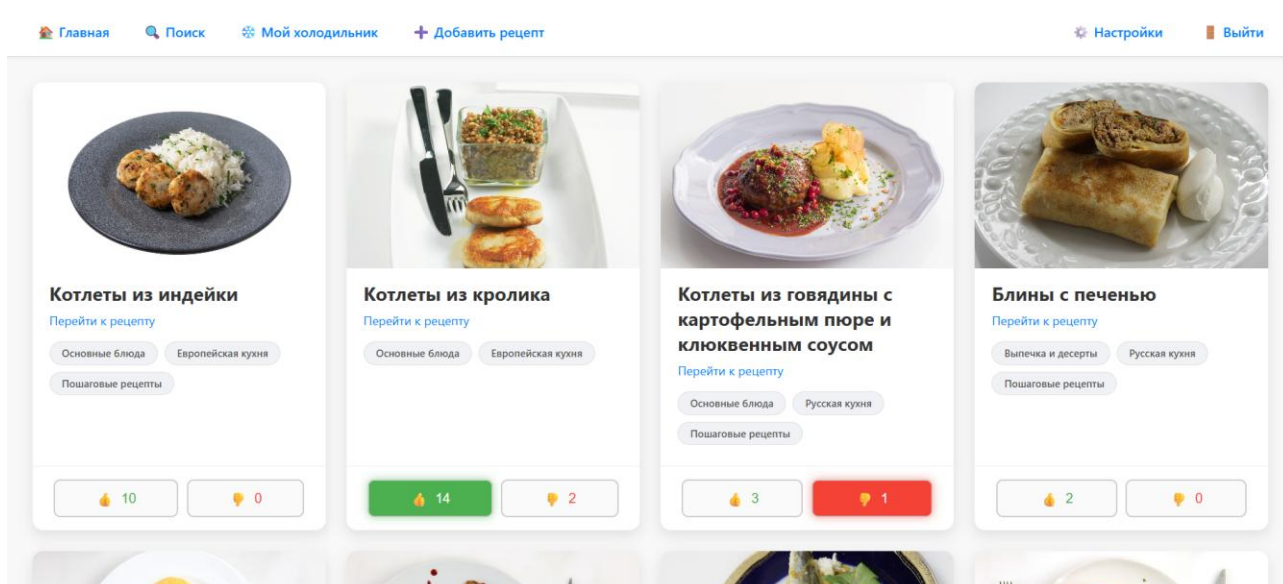


Рисунок 4 — Интерфейс главной странички

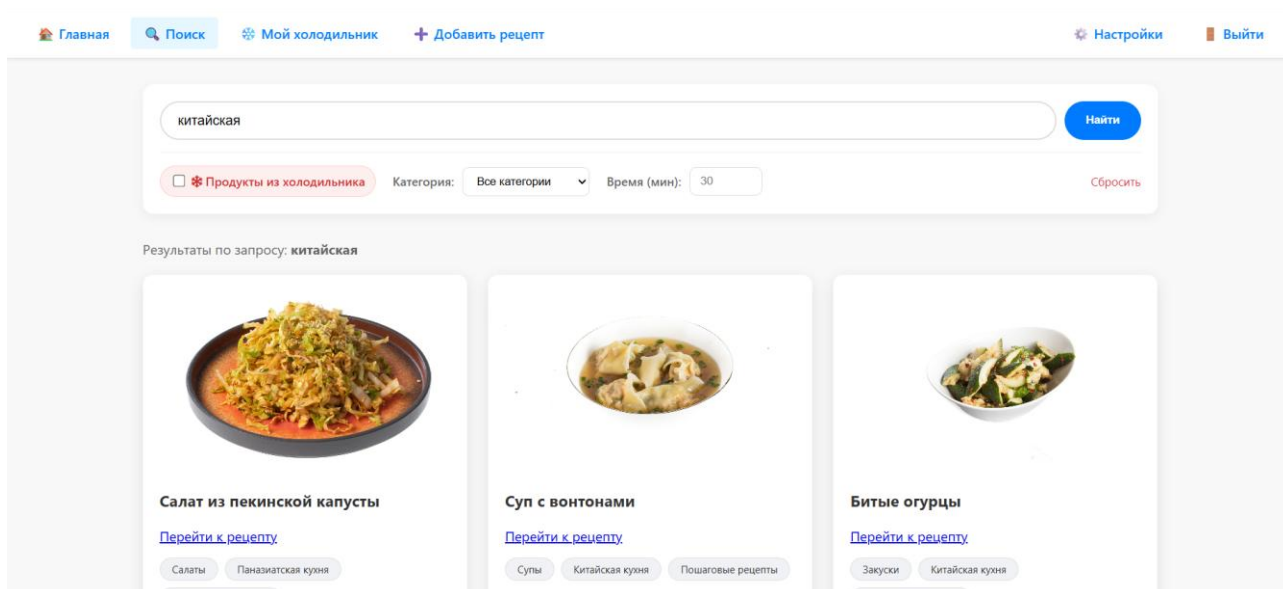


Рисунок 5 — Интерфейс странички поиска

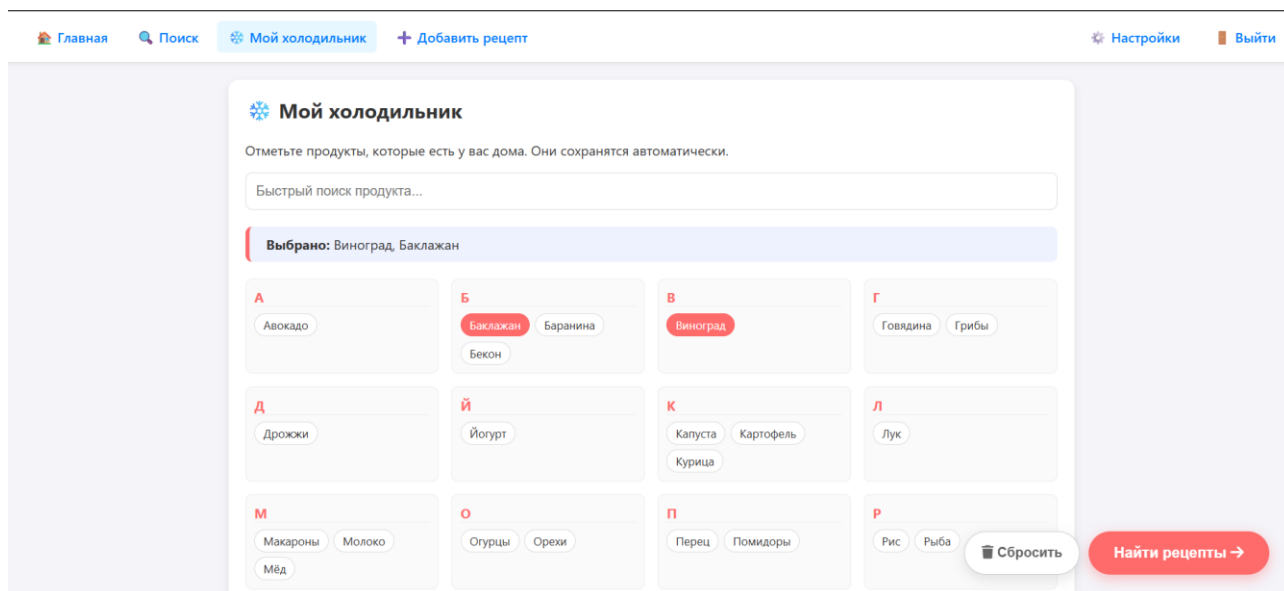


Рисунок 6 — Интерфейс странички «Холодильник»

Профиль и Настройки

Информация об аккаунте

Логин: Bob
Email: bob@mail.ru
ID: 10
Дата регистрации: 30 December 2025

Ваша статистика реакций

5	3	2
Всего взаимодействий	Понравилось	Не понравилось

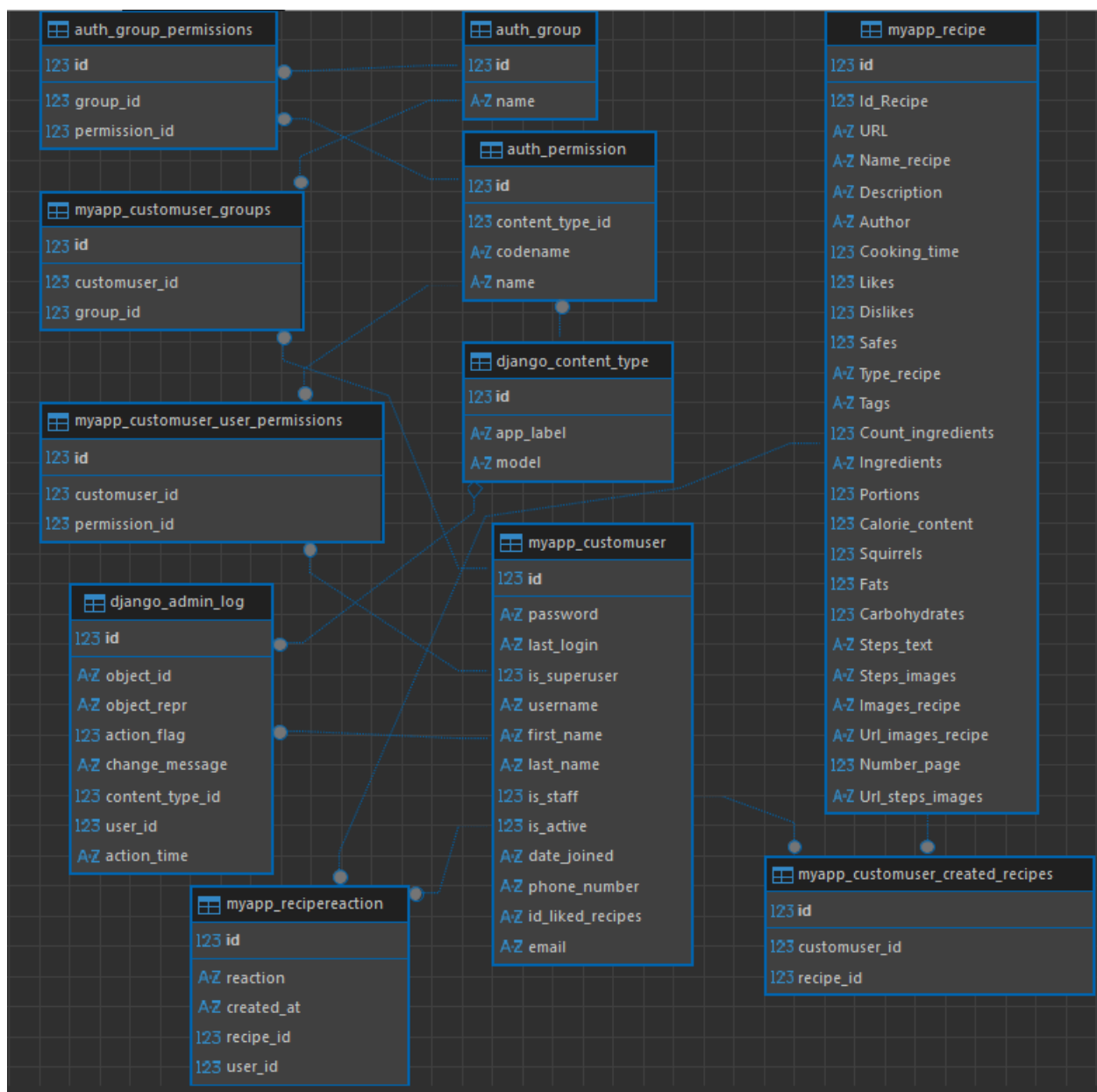
Любимые рецепты (3)

Бефстроганов с белыми грибами и шампиньонами Котлеты из краба Котлеты морковные

Нелюбимые рецепты (2)

Рисунок 7 — Интерфейс странички профиля

Приложение В. Реализация моделей данных приложения



Картинка 8 — Структура ORM-моделей.